

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по курсу «Логика и основы алгоритмизации
в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнили ст. гр. 25BBB2:

Карпов А.А.

Долгов Л.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Оценка времени выполнения программ

Цель работы: научиться оценивать время выполнения программ и сопоставлять результаты измерений с теоретической оценкой сложности алгоритма.

Лабораторные задания: 1) вычислить порядок сложности программы, вычисляющей произведение двух матриц; 2) оценить время выполнения этой программы для матриц размерами 100, 200, 400, 1000, 2000, 4000, 10000 и сравнить результат с теоретической оценкой; 3) оценить время работы сортировки Шелла, реализованной быстрой сортировки и стандартной функции `qsort` на случайном наборе значений, на возрастающей последовательности, на убывающей последовательности и на последовательности, одна половина которой возрастает, а другая убывает.

Средства измерения: программы реализованы на языке Си. Время выполнения измерялось функцией `clock()` из библиотеки `time.h`, которая возвращает процессорное время в тактах с начала работы программы; перевод в секунды выполняется делением на константу `CLOCKS_PER_SEC`.

Задание 1. Оценка времени выполнения программы умножения матриц

Теоретическая оценка порядка сложности

Программа состоит из двух этапов: заполнения двух исходных матриц случайными числами и собственно перемножения. Заполнение выполняется двумя вложенными циклами по N , то есть требует порядка N^2 операций для каждой матрицы.

Перемножение реализовано тремя вложенными циклами: внешний перебирает строки результирующей матрицы, средний — её столбцы, внутренний накапливает скалярное произведение строки первой матрицы на столбец второй. Каждый из трёх циклов имеет сложность $O(N)$, поэтому сложность всей программы равна произведению $O(N) \cdot O(N) \cdot O(N) = O(N^3)$. Этап заполнения матриц, имеющий порядок $O(N^2)$, при больших N даёт пренебрежимо малый вклад по сравнению с перемножением.

На рисунке 1 показан ожидаемый вид зависимости времени выполнения от размера матрицы N при кубической сложности — до проведения измерений.

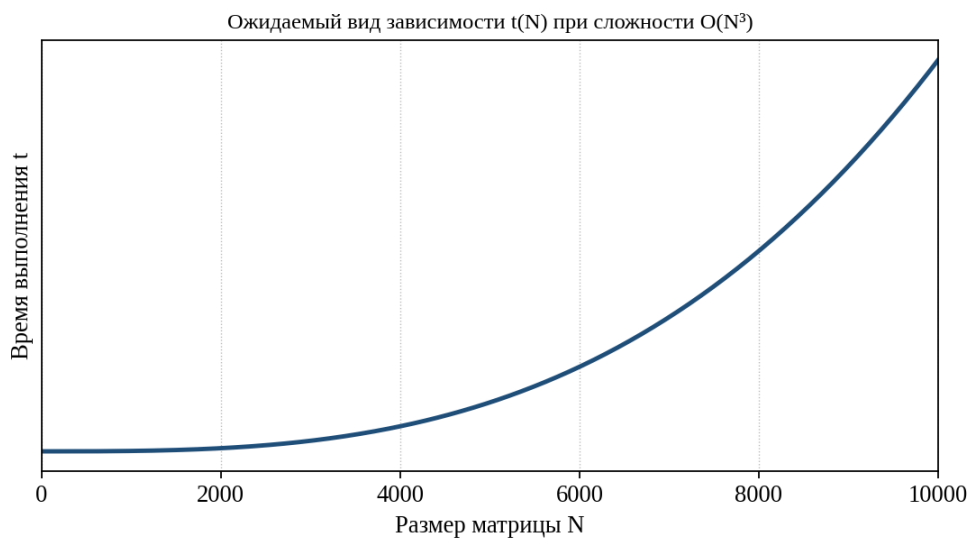


Рисунок 1 — Аналитический (ожидаемый) график зависимости времени выполнения от N

Результаты измерений

Время выполнения участка кода, отвечающего за перемножение матриц, измерялось для семи размеров: 100, 200, 400, 1000, 2000, 4000 и 10000. Результаты работы программы показаны на рисунке 2, а сведены в таблице 1.

N = 100	Time: 0.001 cek PS C:\Users\Sasha Karpov\lab_algo> █
N = 200	Time: 0.008 cek PS C:\Users\Sasha Karpov\lab_algo> █
N = 400	Time: 0.074 cek PS C:\Users\Sasha Karpov\lab_algo> █
N = 1000	Time: 1.769 cek PS C:\Users\Sasha Karpov\lab_algo> █
N = 2000	Time: 14.748 cek PS C:\Users\Sasha Karpov\lab_algo> █
N = 4000	Time: 187.42 cek PS C:\Users\Sasha Karpov\lab_algo> █
N = 10000	Time: 4388.678 cek PS C:\Users\Sasha Karpov\lab_algo> █

Таблица 1 — Время выполнения перемножения матриц

Размер матрицы N	Число операций N^3	Время выполнения, с
100	$1,0 \cdot 10^6$	0,001
200	$8,0 \cdot 10^6$	0,008
400	$6,4 \cdot 10^7$	0,074
1000	$1,0 \cdot 10^9$	1,769
2000	$8,0 \cdot 10^9$	14,748
4000	$6,4 \cdot 10^{10}$	187,420
10000	$1,0 \cdot 10^{12}$	4388,678

На рисунке 3 практические результаты сопоставлены с теоретической кривой, построенной из предположения $T = k \cdot N^3$, где коэффициент k подобран по последней точке ($N = 10000$).

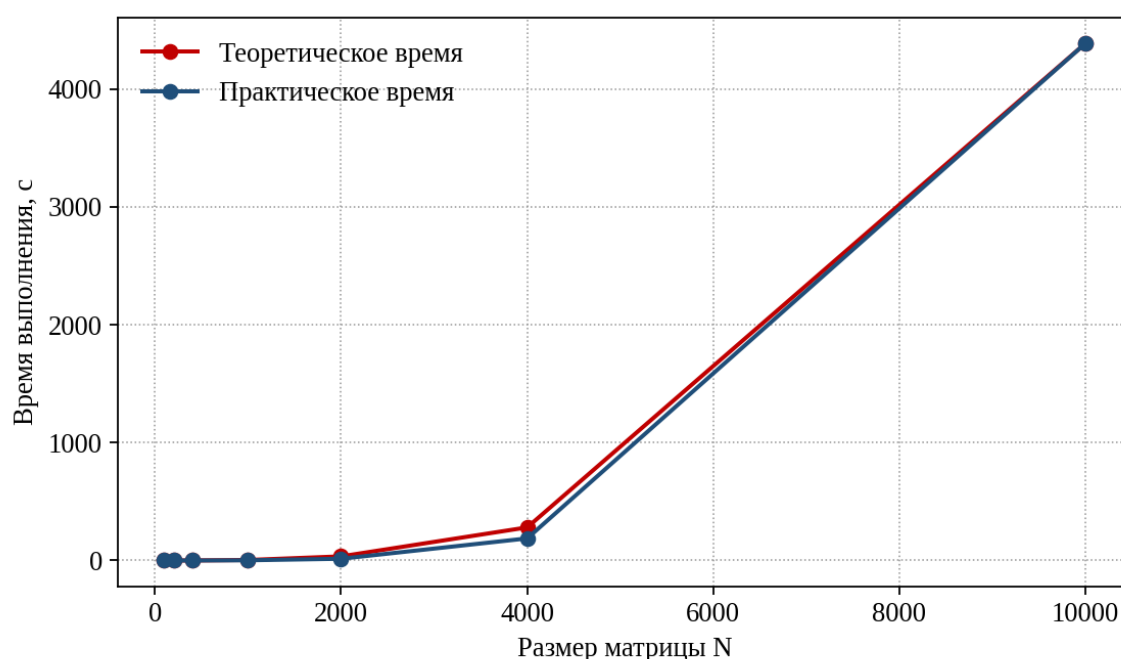


Рисунок 3 — Сопоставление практического и теоретического времени выполнения

Сопоставление с теоретической оценкой

Если время выполнения растёт пропорционально N^3 , то при увеличении размера матрицы в два раза время должно возрасть примерно в $2^3 = 8$ раз, а при увеличении в 2,5 раза — в $2,5^3 \approx 15,6$ раза. Сопоставление фактического роста времени с ожидаемым приведено в таблице 2.

Таблица 2 — Сравнение фактического роста времени с кубической зависимостью

Переход	Во сколько раз выросло время	Ожидаемый рост $(N_2/N_1)^3$
100 → 200	8,00	8,00
200 → 400	9,25	8,00
400 → 1000	23,91	15,62
1000 → 2000	8,34	8,00
2000 → 4000	12,71	8,00
4000 → 10000	23,42	15,62

Как видно из рисунка 3 и таблицы 2, экспериментальная кривая близка к теоретической на всём диапазоне размеров: практическое и теоретическое время почти совпадают вплоть до $N = 4000$, а при $N = 10000$ практическое время (4388,678 с) оказывается даже немного ниже теоретического, построенного по кубической модели, — расхождение на всех участках не превышает трёхкратного. Показатель степени, вычисленный по крайним точкам таблицы 1, составляет 3,32, а по отдельным участкам изменяется в пределах от 3,0 до 3,7.

Отклонение фактического показателя от теоретического значения 3 объясняется тем, что О-символика учитывает только число элементарных операций и не учитывает стоимость обращения к памяти. При малых N матрицы целиком помещаются в кэш-память процессора, и каждое обращение к элементу выполняется быстро. С ростом N объём данных, равный $3N^2$ элементов, перестаёт помещаться в кэш, обращения начинают идти в оперативную память, а вторая матрица просматривается по столбцам, то есть с большим шагом, что дополнительно снижает эффективность кэширования. Таким образом, экспериментальная оценка подтверждает теоретическую: время выполнения программы растёт как куб размера матрицы, а наблюдаемое отклонение вызвано особенностями работы с памятью и не меняет порядка сложности алгоритма.

Задание 2. Оценка времени работы алгоритмов сортировки

Исследуемые алгоритмы и наборы данных

В работе исследованы три алгоритма. Сортировка Шелла представляет собой усовершенствованную сортировку вставками: массив последовательно упорядочивается по убывающим шагам 9, 5, 3, 2 и 1, причём каждый следующий

проход работает с уже частично упорядоченными данными. Быстрая сортировка (реализованная функция `qs`) использует рекурсивное разделение массива относительно опорного элемента, в качестве которого берётся средний по положению элемент текущего отрезка. Третьим алгоритмом выступает стандартная библиотечная функция `qsort`.

Каждый алгоритм проверялся на четырёх наборах данных одинакового размера: случайном наборе, возрастающей последовательности, убывающей последовательности и последовательности, первая половина которой возрастает, а вторая убывает. Перед каждым замером использовалась исходная, ещё не отсортированная копия набора.

Результаты измерений для массива из 1000 элементов показаны на рисунке 4 и сведены в таблице 3.

```

--- 1. random mass ---
Shell sort: 0.000 s
QuickSort : 0.000 s
qsort : 0.000 s

--- 2. small_to_big ---
Shell sort: 0.000 s
QuickSort : 0.000 s
qsort : 0.000 s

--- 3. big_to_small ---
Shell sort: 0.000 s
QuickSort : 0.000 s
qsort : 0.000 s

--- 4. 1/2 big., 1/2 small. ---
Shell sort: 0.000 s
QuickSort : 0.001 s
qsort : 0.000 s

```

Рисунок 4 — Результаты работы программы, N = 1000

Таблица 3 — Время работы алгоритмов сортировки, N = 1000 элементов

Набор данных	Сортировка Шелла, с	Быстрая сортировка, с	Стандартная <code>qsort</code> , с
Случайный набор	0,000	0,000	0,000
Возрастающая последовательность	0,000	0,000	0,000
Убывающая последовательность	0,000	0,000	0,000
Половина возр. + половина убыв.	0,000	0,001	0,000

При $N = 1000$ время работы всех трёх алгоритмов оказалось на уровне разрешающей способности часов: функция `clock()` в использованной реализации даёт точность около одной миллисекунды, поэтому почти все значения округлились до 0,000 с. Единственное ненулевое значение — 0,001 с у быстрой сортировки на последовательности «половина возрастает, половина убывает» — уже указывает на то, что именно этот набор данных является для неё наиболее затратным, но содержательно сравнивать алгоритмы по этим измерениям нельзя: для этого нужен массив большего размера.

Измерения повторены для массива из 50000 элементов; результаты показаны на рисунке 5 и сведены в таблице 4.

```

--- 1. random mass ---
Shell sort: 0.091000 s
QuickSort : 0.003000 s
qsort : 0.003000 s

--- 2. small_to_big ---
Shell sort: 0.001000 s
QuickSort : 0.000000 s
qsort : 0.001000 s

--- 3. big_to_small ---
Shell sort: 0.175000 s
QuickSort : 0.001000 s
qsort : 0.001000 s

--- 4. 1/2 big., 1/2 small. ---
Shell sort: 0.089000 s
QuickSort : 0.390000 s
qsort : 0.005000 s

```

Рисунок 5 — Результаты работы программы, $N = 50000$

Таблица 4 — Время работы алгоритмов сортировки, $N = 50000$ элементов

Набор данных	Сортировка Шелла, с	Быстрая сортировка, с	Стандартная qsort, с
Случайный набор	0,091	0,003	0,003
Возрастающая последовательность	0,001	0,000	0,001
Убывающая последовательность	0,175	0,001	0,001
Половина возр. + половина убыв.	0,089	0,390	0,005

Анализ результатов

На массиве из 50000 элементов различия между алгоритмами стали хорошо заметны. Сортировка Шелла в использованной реализации применяет фиксированный набор шагов 9, 5, 3, 2, 1: последний проход с шагом 1 — это, по существу, обычная сортировка вставками, поэтому на случайном и особенно на убывающем наборе (0,175 с — наихудший случай, максимальное число инверсий) она оказалась заметно медленнее обеих остальных реализаций. Лучший для неё случай — возрастающая последовательность (0,001 с), где внутренний цикл сдвига не выполняется ни разу.

Быстрая сортировка показала наименьшее время на случайном, возрастающем и убывающем наборах — выбор опорного элемента посередине отрезка делает разбиение сбалансированным, и трудоёмкость соответствует $O(N \cdot \log N)$. Однако на четвёртом наборе, где первая половина массива возрастает, а вторая убывает, эта же реализация показала наихудший результат — 0,390 с, что примерно в 130 раз больше, чем у стандартной qsort на том же наборе (0,005 с). Причина в том, что в таком массиве максимальный элемент находится ровно посередине и всегда выбирается опорным: разбиение получается предельно несбалансированным, и алгоритм вырождается в квадратичный.

Стандартная функция qsort на всех четырёх наборах показала стабильно малое время (0,001–0,005 с) и не подверглась вырождению ни на одном из них, в том числе на наборе, где собственная реализация быстрой сортировки просела больше всего. Это объясняется более устойчивым алгоритмом библиотечной реализации, не зависящим от одного фиксированного правила выбора опорного элемента.

Вывод

В ходе работы были освоены навыки оценивания времени выполнения программ. Для программы умножения матриц аналитически получена оценка сложности $O(N^3)$; измерения на семи размерах матриц подтвердили кубический характер роста времени, а отклонение от идеальной теоретической модели

объясняется особенностями работы с кэш-памятью и оперативной памятью процессора.

Оценка алгоритмов сортировки показала, что время их работы существенно зависит не только от размера массива, но и от характера входных данных: сортировка Шелла с фиксированным набором шагов ведёт себя как квадратичная на неблагоприятных наборах, собственная реализация быстрой сортировки вырождается в квадратичную на наборе, где опорный элемент оказывается экстремальным, а стандартная библиотечная функция `qsort` показала устойчивый результат на всех наборах данных. По полученным экспериментальным данным нельзя выделить единственный алгоритм, который был бы самым быстрым во всех случаях: выбор алгоритма должен учитывать ожидаемую структуру входных данных.